Bioinformatics, 34(3), 2018, 388–397
doi: 10.1093/bioinformatics/btx609
Advance Access Publication Date: 29 September 2017
Original Paper

OXFORD

Genome analysis

A new haplotype block detection method for dense genome sequencing data based on interval graph modeling of clusters of highly correlated SNPs

Sun Ah Kim[1], Chang-Sung Cho[2], Suh-Ryung Kim[2], Shelley B. Bull[3,4] and Yun Joo Yoo[2,5,*]

[1]Department of Statistics and [2]Department of Mathematics Education, Seoul National University, Seoul 08826, South Korea, [3]Prosserman Centre for Health Research, The Lunenfeld-Tanenbaum Research Institute, Sinai Health System, Toronto M5T 3L9, Canada, [4]Division of Biostatistics, Dalla Lana School of Public Health, University of Toronto, Toronto M5T 3M7, Canada and [5]Interdisciplinary Program in Bioinformatics, Seoul National University, Seoul 08826, South Korea

*To whom correspondence should be addressed.

Associate Editor: John Hancock

Received on December 15, 2016; revised on September 11, 2017; editorial decision on September 20, 2017; accepted on September 28, 2017

Abstract

Motivation: Linkage disequilibrium (LD) block construction is required for research in population genetics and genetic epidemiology, including specification of sets of single nucleotide polymorphisms (SNPs) for analysis of multi-SNP based association and identification of haplotype blocks in high density sequencing data. Existing methods based on a narrow sense definition do not allow intermediate regions of low LD between strongly associated SNP pairs and tend to split high density SNP data into small blocks having high between-block correlation.

Results: We present Big-LD, a block partition method based on interval graph modeling of LD bins which are clusters of strong pairwise LD SNPs, not necessarily physically consecutive. Big-LD uses an agglomerative approach that starts by identifying small communities of SNPs, i.e. the SNPs in each LD bin region, and proceeds by merging these communities. We determine the number of blocks using a method to find maximum-weight independent set. Big-LD produces larger LD blocks compared to existing methods such as MATILDE, Haploview, MIG++, or S-MIG++ and the LD blocks better agree with recombination hotspot locations determined by sperm-typing experiments. The observed average runtime of Big-LD for 13 288 240 non-monomorphic SNPs from 1000 Genomes Project autosome data (286 East Asians) is about 5.83 h, which is a significant improvement over the existing methods.

Availability and implementation: Source code and documentation are available for download at http://github.com/sunnyeesl/BigLD.

Contact: yyoo@snu.ac.kr

Supplementary information: Supplementary data are available at *Bioinformatics* online.

© The Author 2017. Published by Oxford University Press. All rights reserved. For Permissions, please e-mail: journals.permissions@oup.com

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

1 Introduction

Understanding the patterns of linkage disequilibrium (LD) across the human genome, especially its block-like structures, has interested many researchers in genetics because it provides useful insight for disease association mapping and population genetics (Barrett *et al.*, 2005; Daly *et al.*, 2001; Gibbs *et al.*, 2003; Jeffreys *et al.*,2001; Reich *et al.*, 2001; Sabeti *et al.*, 2007; Slatkin, 2008; Twells *et al.*, 2003; Wang *et al.*, 2002). These blocks are called LD blocks or haplotype blocks. It has been observed that LD block regions can vary in size, and strong LD extends over each block region until it breaks down abruptly possibly due to recombination hotspots or population genetic phenomenon (Jeffreys *et al.*, 2001; Stephens and Scheet, 2005). Recombination, mutation, selection or other evolutionary history can affect LD block patterns in the aspects of location and length of blocks (McVean *et al.*, 2004). Identifying LD blocks, therefore, can provide evidential groundings for population genetic arguments and necessary information for the design and analysis of genetic association studies for complex diseases (Rana *et al.*, 2004; Wang *et al.*, 2002; Wu *et al.*, 2010).

Researchers have proposed several methods to determine LD blocks from population data consisting of independent unrelated individuals (Barrett *et al.*, 2005; Gabriel *et al.*, 2002; Pattaro *et al.* 2008; Taliun *et al.*, 2014; Wang *et al.*, 2002). Pattaro *et al.* (2008) developed a method, called MATILDE, to identify LD blocks based on LD measures either D' (Lewontin, 1964) or r^2 using a MCMC algorithm. Gabriel *et al.* (2002) showed that the human genome can be partitioned into LD blocks within which little evidence of recombination and less haplotype diversity is observed. They proposed a method, called the confidence interval (CI) method, to define haplotype blocks using a 95% confidence bound for an LD measure D'. Wang *et al.* (2002) suggested a different approach to defining LD blocks using the four-gamete test (FGT) that identifies the occurrence of past recombination events. Both methods, CI and FGT, are implemented in the software package Haploview (Barrett *et al.*, 2005) which is a visualization and analysis tool for haplotype patterns. In Haploview, LD blocks can be recognized by an additional algorithm called 'Solid Spine (SS)' with which an LD block consists of all the SNPs that are in strong LD with the first and last SNPs in the block (Haploview documentation). Recently, Taliun *et al.* (2014) developed an algorithm called MIG++, with increased computational efficiency incorporating an incremental computation strategy and estimation methods of approximated confidence interval for D' (Zapata, 2011) applying the LD block definition of Gabriel *et al.* (2002) to extensive high-throughput datasets such as whole-genome sequencing data. Another computationally improved LD block construction algorithm named S-MIG++, also using the definition of Gabriel *et al.* (2002), adopts a two-step approach: the estimation step finding the upper limits of the haplotype blocks by sampling a fraction of SNP pairs and the refinement step determining exact haplotype boundaries (Taliun *et al.*, 2016).

Nowadays, high-throughput genotype data such as genome-wide SNP array data or whole genome/exome sequencing data are collected and analyzed for gene-trait association studies (Manolio, 2010; Wang *et al.*, 2015). An efficient approach for the analysis of high-throughput genomic data and a good alternative to single-SNP methods is to combine the effects of multiple SNPs on disease phenotype, (Neale and Sham, 2004; Neale *et al.*, 2011; Pan, 2009; Wu *et al.*, 2010). However, to apply multi-SNP analysis methods, it is necessary to specify a set of SNPs to be combined prior to performing multi-SNP tests. To specify these SNP sets, LD block information can be utilized in combination with gene or pathway information (Wu *et al.*, 2010; Zhao *et al.*, 2011). For some multi-SNP tests, higher LD among combined SNPs can improve the power of the test, but having fewer SNP-set analysis units and independent global null hypotheses that correspond to each of the units is preferred as a simple way to reduce type I error inflation (Derkach *et al.*, 2014; Yoo *et al.*, 2015). Yoo *et al.* (2015; 2017) proposed and evaluated a gene-based multi-SNP combination method that combines the effect of SNPs in 'LD bins' using CLQ, a *clique*-based SNP clustering algorithm.

In this paper, we develop a new algorithm called Big-LD that adopts a wider sense LD block definition—a block region based on the LD bin clusters. To do this we utilize LD bins constructed using CLQ-D, a modified version of the previous LD bin construction algorithm CLQ (Yoo *et al.*, 2015, 2017), and consider the intervals defined by the physical positions of the starting and ending SNPs of each LD bin. Each interval corresponding to an LD bin represents the physical range of LD associated with SNPs in that bin. In the Big-LD algorithm, we model the relationships between intervals by a graph with vertices that represent the LD bin intervals and edges that represent the overlap between LD bin intervals. By applying an algorithm that partitions the vertex set of the interval graph into mutually exclusive sets by selecting an independent set with the maximum weight, Big-LD automatically determines the number of partitions to be formed into LD blocks. Using data from the 1000 Genomes Project and the HapMap Phase III East Asian populations, we show that the LD blocks found by Big-LD are larger than the blocks produced by the existing methods MATILDE, Haploview, MIG++ or S-MIG++. Furthermore, using data of class II regions of MHC with true recombination hotspot information experimentally determined in a prior study (Jeffreys *et al.*, 2000, 2001), we show that, compared to the existing methods, the LD block boundaries obtained by Big-LD coincide more closely with recombination hotspot positions.

2 Materials and methods

2.1 Terminology in basic graph theory

A *graph* G is an ordered pair (V, E), where $V = \{v_1, v_2, \ldots, v_n\}$ is a set of vertices, E is a set of edges which is a subset of the set of unordered pairs of vertices of G. If v_i and v_j are vertices that are incident with an edge, i.e. $e = \{v_i, v_j\} \in E$, then it is said that v_i and v_j are *adjacent*. The number of edges incident to a vertex v of a graph G is called the *degree* of v. The set of vertices adjacent to a vertex v of a graph is called the *neighborhood* of v. A graph H is said to be a *subgraph* of a graph G if the edge set and the vertex set of H are a subset of the edge set and a subset of the vertex set of G, respectively. A subgraph H of G is said to be *induced* by a vertex set S of G if two vertices in H are adjacent in H whenever they are adjacent in G.

Let $I = \{I_1, I_2, \ldots, I_n\}$ be a set of closed intervals of real lines. The *interval graph* G of I, denoted by G_I, is defined as a graph (V_I, E_I) where $V_I = \{v_1, \ldots, v_n\}$ and $\{v_i, v_j\} \in E_I$ if and only if $I_i \cap I_j \neq \varnothing$.

Given a graph $G = (V, E)$, let w_E be a function from E to $\mathbb{R}$. For each edge e, $w_E(e)$ is called the *weight* of e. Together with weights on its edges, G is said to be an *edge-weighted graph* and denoted by (V, E, w_E). Given a function from V to $\mathbb{R}$, $w_V(v)$ is called the *weight* of v for each vertex v and together with weights on its vertices, G is said to be a *vertex-weighted graph* and denoted by (V, E, w_V).

A set of vertices is called a *clique* if any two distinct vertices in the set are connected by an edge. A *maximal clique* is a clique that is

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

not included in any other clique. A *largest clique* is a clique that has the maximum size among all the cliques in the graph.

A set of vertices is said to be an *independent set* if no two vertices in the set are adjacent. In a vertex-weighted graph $G = (V, E, w_V)$, *maximum weight independent set* is an independent set where the sum of weights on vertices in the set is greatest among the sum of weighs of all the independent set of G.

2.2 CLQ-D: construction of LD bins

Suppose that the SNP sequence data consist of genotypes of a total of n SNPs in m individuals. Let $S = \{s_1, s_2, \ldots, s_n\}$ be the set of SNPs and $y(s_i)$ be the physical position (in base pair; bp) of s_i in the chromosome. We may assume that $s_1, s_2, \ldots, s_n$ are arranged so that $y(s_1) < \ldots < y(s_n)$. We also assume an additive genetic model such that the genotype of each individual for each SNP is the count of the risk allele (usually the minor allele), i.e. either of 0, 1 or 2. Each genotype vector of SNP s_i of m individuals is denoted as X_i. Then the pairwise LD measure r^2_{ij} between two SNPs s_i and s_j can be obtained by squaring the Pearson correlation coefficient r_{ij} between additive genotypes X_i and X_j. We can construct an edge-weighted complete graph G with the given SNP set S as the vertex set and the edge set $E = \{\{s_i, s_j\} \mid s_i, s_j \in V,\ i \neq j\}$ with $w_E(\{s_i, s_j\}) = |r_{ij}|$ for $\{s_i, s_j\} \in E$. Then the SNPs and their LD structures are represented by the graph $G = (S, E, w_E)$.

Overall, LD between two SNPs tends to decrease as the physical distance between them increases (Collins *et al.*, 1999; Ott, 2000; Takahata, 1982). However, this relationship does not occur monotonically, and a group of strongly correlated SNPs can be formed by non-consecutive SNPs. We call a group of strongly correlated SNPs an *LD bin* within which every SNP is highly correlated with the other SNPs. In a previous study, we developed an algorithm called CLQ that partition SNPs into LD bins using a greedy clique partitioning method (Yoo *et al.*, 2015). The CLQ algorithm starts by modeling SNP set data by a graph in which vertices are SNPs and edges between two SNPs s_i and s_j exist whenever $|r_{ij}|$ between them exceeds a given threshold (θ). Then, by the Bron-Kerbosch algorithm (Bron and Kerbosch, 1973; Eppstein *et al.*, 2010), all maximal cliques in the graph are identified and among them the largest clique is taken as the first LD bin. We then remove SNPs already taken as the LD bin, and again choose the next largest clique as the next LD bin among remaining maximal cliques. Repeating this procedure, the CLQ algorithm partitions the SNPs into a set of LD bins that are actually cliques in the graph.

For the new LD block partition algorithm, we modify the original CLQ LD bins selection to give priority to bins where SNPs are relatively close in physical proximity. To do this, we develop CLQ-D as a modified version of CLQ incorporating a marker-density function of cliques defined using the size k of the clique and the range of physical positions of the SNPs in the clique. The marker-density function D of a clique Q with size k is defined as $D(Q) = \frac{k}{b-a}$ if $k > 1$, where a and b are the minimum and the maximum, respectively, of the bp positions of the SNPs in Q, and $D(Q) = 0$ if $k = 1$. When choosing an LD bin among the set of maximal cliques in each iteration of the greedy algorithm, we give the priority to the clique with the maximum marker-density D. Details of the newly developed CLQ-D algorithm are given in Supplementary Methods.

2.3 Big-LD: construction of LD blocks

Let $H = \{B_1, B_2, \ldots, B_l\}$ be the set of LD bins obtained by the CLQ-D algorithm (Fig. 1B). Interval representation of H can be formulated from the physical positions (bp) of SNPs. For each LD bin

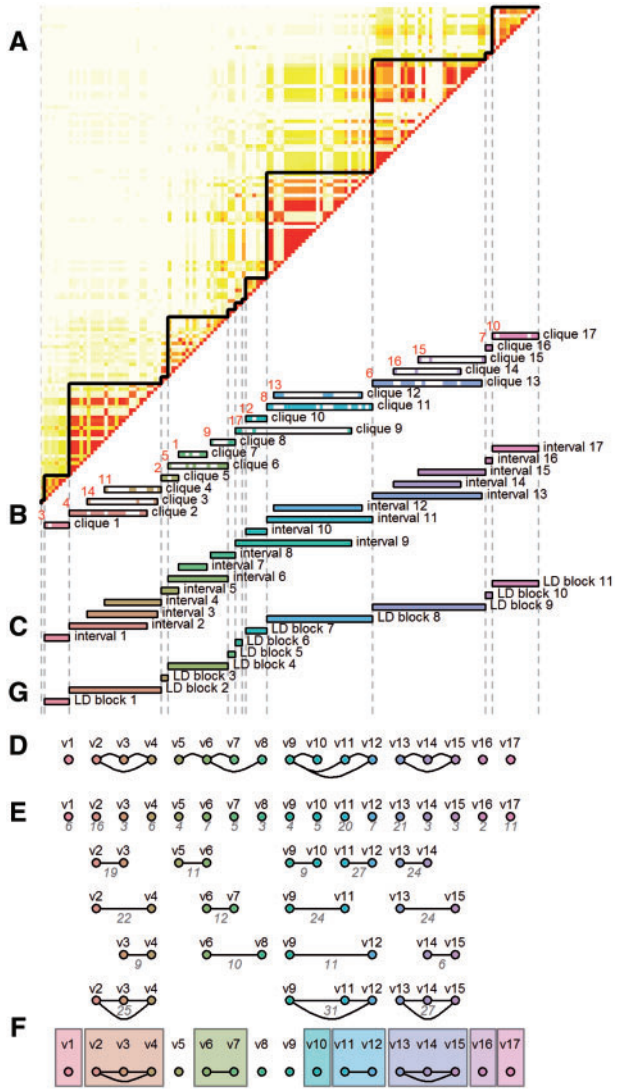


Fig. 1. An illustration of CLQ-D and BIG-LD algorithm applied to a region in chromosome 22 (chr 22: 20 197 335–20 277 113) using 1000G dataset. (**A**) An LD heatmap of 141 SNPs in this region. (**B**) LD bins constructed using CLQ-D algorithm in this region. The positions of SNPs in each LD bin region are shown in color in each strip. The numbers in red color show the order of LD bins selected by the greedy process of CLQ-D. (**C**) Interval representation of LD bins obtained in B. (**D**) The interval graph G of the set of intervals given in C. Each vertex corresponds to LD bin in B and each edge corresponds to the overlapping relationship between two intervals. (**E**) All possible cliques in G. The weight on each interval clique (grey colored number) is obtained as the total number of SNPs in the union of LD bins that correspond to the vertices of each clique in G. (**F**) The result of the first iteration of BIG-LD algorithm. For each chosen clique in G, union of intervals which correspond to vertices in the clique is taken as an LD block. (**G**) The final LD blocks obtained by BIG-LD algorithm

$B_i = \{s_{i_1}, s_{i_2}, \ldots, s_{i_{k_i}}\}$ where $y(s_{i_1}) < y(s_{i_2}) < \ldots < y\left(s_{i_{k_i}}\right)$, we can define an interval corresponding to B_i as $I_i = [y(s_{i_1}), y(s_{i_{k_i}})]$. We then obtain a set of intervals $\mathcal{I} = \{I_1, I_2, \ldots, I_l\}$ such that I_i corresponds to B_i for each $i = 1, \ldots, l$. (Fig. 1C).

Each interval corresponding to an LD bin represents the range of the LD associated with SNPs in that bin. When some LD bin intervals overlap one another, we may infer that the LD extends over the combined region of these overlapping intervals. In this sense, we assume that a set of LD bin intervals that overlap each other are more

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

likely to be included in the same LD block. Based on this assumption, we construct an interval graph which reflects the intersection patterns of the range of LD bins and cluster them into an optimal partition where the LD within the same block remains strong and the LD between the SNPs in different blocks is relatively low. To do this, first define an interval graph $G = (V, E)$ of $\mathcal{I}$ with the vertex set $V = \{v_1, v_2, \ldots, v_l\}$ and the edge set $E = \{\{v_i, v_j\} | I_i \cap I_j \neq \varnothing\}$. (Fig. 1D).

A set of mutually overlapping intervals corresponds to a clique in the interval graph G, and the union of the intervals corresponding to the clique forms an interval that could be considered as an LD block since strong LD extends over these overlapping intervals. By an appropriate clique partitioning of the interval graph G, we can find an optimized LD block partitioning of the genomic data. We perform this clique partitioning using a greedy algorithm which finds a maximum weight independent set of a graph which is updated at each iteration by taking the cliques in the current interval graph as the vertices and joining two vertices by an edge whenever the intervals corresponding to the two vertices overlap in the current interval graph. For the vertex weights of this graph, we assign the number of SNPs in the clique corresponding to the vertex (Fig. 1E). In this weighted graph constructed using all possible cliques of the interval graph, we choose an independent set of vertices such that the sum of weights assigned to its elements is the maximum. Each interval region corresponding to a vertex in this set is recruited as an LD block, and LD blocks selected in this way would not overlap each other since the vertices in an independent set do not have edges between them (Fig. 1F). Once we recruit LD blocks from one iteration of the greedy algorithm, some SNPs could remain that are not covered by the intervals corresponding to the independent sets we have chosen so far. We apply the same process to the graph of updated intervals after deleting the chosen SNPs from the previously selected intervals.

The detailed steps of Big-LD algorithm are as follows:

Step 1. Find all the cliques, say $C_1, C_2, \ldots, C_M$, in an interval graph $G = (V, E)$ of $\mathcal{I}$ by using the algorithm of Tsukiyama *et al.* (1977) implemented in the R igraph package (Csardi and Nepusz, 2006).

Step 2. For each $i \in \{1, \ldots, M\}$, since the intervals corresponding to the vertices in C_i are mutually overlapping, the union of those intervals forms an interval which we denote by J_i.

Step 3. Take an interval graph of $\mathcal{J}^{(1)} = \{J_1^{(1)}, \ldots, J_M^{(1)}\}$ with the vertex set $V^{(1)} = \{v_1^{(1)}, \ldots, v_M^{(1)}\}$. We assign a weight to vertex $v_i^{(1)}$ as the number of all SNPs which belong to the LD bins corresponding to clique C_i for each $i = 1, \ldots, M$ to obtain a weighted interval graph.

Step 4. By using an algorithm to find a maximum weighted independent set of a vertex weighted interval graph (the details of the algorithm can be found in Supplementary Methods), find a maximum weight independent set $K^{(1)} = \{v_{i_1}^{(1)}, \ldots, v_{i_{k_1}}^{(1)}\}$ of the weighted interval graph obtained in Step 3. Note that the intervals $J_{i_1}^{(1)}, \ldots, J_{i_{k_1}}^{(1)}$ corresponding to the vertices in $K^{(1)}$ are mutually disjoint. We take $J_{i_1}^{(1)}, \ldots, J_{i_{k_1}}^{(1)}$ as the first set of LD blocks obtained from the first iteration.

Step 5. Update each $J_i^{(1)}$, $i = 1, \ldots, M$, by removing the region taken as the LD block at the previous step, i.e. the difference set after removing the union of $J_{i_1}^{(1)}, \ldots, J_{i_{k_1}}^{(1)}$. It could become an empty set or a non-empty subinterval of $J_i^{(1)}$. Update $\mathcal{J}^{(1)}$ into $\mathcal{J}^{(2)}$ as the set of the resulting subintervals.

Step 6. If $\mathcal{J}^{(2)}$ is non-empty, then repeat the same procedure from Step 3 to Step 6 for $\mathcal{J}^{(2)}$ to obtain a set of LD blocks which are to be added to the previous set of LD blocks to form the current set of LD blocks. Repeat the iteration until the updated set of intervals is empty. If it is empty set, stop and obtain the final set of LD blocks (Fig. 1G).

To apply the CLQ-D and the Big-LD algorithms to big data with many SNPs, we can partition the entire region of the data into sub-regions of consecutive SNPs and apply the CLQ-D and Big-LD separately to reduce the computing time and memory. To do that, we need some strategy to efficiently combine the results of these sub-tasks. In the implementation of the Big-LD algorithm to deal with data with many SNPs, we predefine an upper limit for sub-task size (λ), the number of SNPs in a sub-region, and divide the genome by finding some 'weak LD points' to make each sub-region smaller than the size limit and the sub-region break points at relatively weak LD regions. The details of the rules to divide the SNPs into sub-tasks are stated in Supplementary Methods.

Since the probability to detect LD is low when the minor allele frequency (MAF) of a SNP in consideration is low (Goddard *et al.*, 2000; Lewontin, 1995), we basically developed the Big-LD algorithm to construct LD blocks based on common SNPs (MAF ≥ 0.05). For the data including rare/low frequency SNPs, we devised a two-stage process in which an initial LD block construction is first performed using the SNPs of which MAF is at least a threshold value, followed by extra steps that assign rare/low frequency SNPs with the MAF below the threshold into initially constructed blocks or add new blocks. The details of these extra steps are stated in Supplementary Methods.

2.4 Evaluation data

We conducted performance evaluations and comparisons with some existing methods using three datasets: 1000 Genomes Project phase 1 release 3 (1000G) with 286 individuals from JPT, CHB and CHS populations (1000 Genomes Project Consortium, 2012); HapMap phase III (HapMap) with 170 individuals from JPT and CHB populations (International HapMap 3 Consortium, 2010); the class II region of the major histocompatibility complex (MHC) with 50 north-European British semen donors (Jeffreys *et al.*, 2001).

To evaluate runtime, memory and the effect of sub-region sizes for the implemented Big-LD algorithm, we used the 1000G dataset of chromosome 1 through 22 (13 288 240 non-monomorphic SNPs). To compare the Big-LD block partition results with the other methods and assess recombination hotspot estimation we used the 1000G phased genotype data of 75 582 SNPs in chromosome 22 (chr22: 16 050 612–51 243 297) after trimming with a minor allele frequency threshold of 0.05 and excluding insertion/deletion(indel) polymorphisms. For the HapMap dataset, we used the phased genotype data of 13 994 SNPs in chromosome 22 (chr22: 16 180 203–51 219 006) after applying the same pruning criteria as for 1000G.

The original MHC dataset contains unphased diploid genotypes for 264 SNPs and 22 indels in 84 kb resequenced regions (Jeffreys *et al.*, 2001). In this study, we used a subset of 263 SNPs without missing data or any indels for comparison of the LD block partition methods and the recombination hotspot locations experimentally identified by sperm-typing analysis of the same individuals reported in Jeffreys *et al.* (2001). Since the original data are unphased diploid genotype data, the haplotypes of 263 SNPs were reconstructed by PHASE v2.1.1 program (Stephens *et al.*, 2001; Stephens and Scheet, 2005) and these were used as the input data for various LD block partition methods and recombination hotspot estimation methods. The MHC dataset can be downloaded from http://www.le.ac.uk/genetics/ajj/HLA/.

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

3 Results

3.1 Implementation and performance evaluation

We implemented the CLQ-D and Big-LD algorithms in R. For the Bron-Kerbosch algorithm (Bron and Kerbosch, 1973; Eppstein *et al.*, 2010) that finds all maximal cliques in a graph, required in CLQ-D and Big-LD implementation, we used the R library '*igraph*' package (Csardi and Nepusz, 2006) in which the Bron-Kerbosch algorithm is written in ANSI C. For the algorithm to find all cliques in the graph, we used the method of Tsukiyama *et al.* (1977) implemented in R *igraph*. We set threshold values required to run CLQ-D as $\theta = 0.5$ and $\rho = 40$kb throughout the entire evaluation process, which has been chosen based on the results using various thresholds for θ (see Supplementary Methods and Table S1). For a graph G with n vertices, the algorithm finds all maximal cliques in time $O(dn\,3^{d/3})$ where the value d, the *degeneracy* of a graph G, is the smallest value such that every nonempty subgraph of G contains a vertex of degree at most d. The computational time for a graph that has large degeneracy may grow exponentially. Therefore, we incorporate a heuristic procedure in CLQ-D to reduce computational time when the graph has large degeneracy (see Supplementary Methods). The implemented R functions are packaged into 'BigLD' and can be downloaded from http://github.com/sunnyeesl/BigLD. All experiments and performance evaluations were performed using these R functions on a machine with Intel i7-6700 (3.4 GHz) CPU.

For genomic regions with many SNPs, Big-LD divides the task into several sub-tasks in which the number of SNPs is less than a predetermined limit (λ). We first assessed the effect of the number of SNPs in one sub-task on runtime and memory usage using various samples chosen according to MAF and inter-SNP distance (Supplementary Fig. S1). The runtime and memory required for each region without dividing them into sub-tasks grows exponentially with the number of SNPs (Supplementary Fig. S2 and Table S2). Next, we tested how an upper limit for sub-task partition affects the LD block partition results of Big-LD for λ from 1000 to 3000 (increasing by 500) using the entire chromosome 22 data of 1000G dataset (Supplementary Tables S3 and S4). When the upper limit is smaller, the runtime and memory to partition the data is reduced with little difference in the block partition results. Since more than 99% of the LD block partition results using the upper limit of 1500 and 3000 for sub-task size are the same (Supplementary Table S4) and the run-time and memory to run the same job is 81 and 70% respectively for the upper limit of 1500 instead of 3000, we decided to use 1500 as the upper limit for the number of SNPs in sub-tasks throughout the evaluation process. We also measured the runtime and memory usage of the sub-task partitioning stage for various sub-task size limit (λ) and for the data trimmed using two MAF threshold values of 0.05 and 0.01. The results show that mostly it takes more time to divide the sequence into sub-tasks with smaller λ values and to process the data including more low frequency variants (Supplementary Fig. S3 and Table S5).

To assess the computational feasibility to apply the Big-LD for the entire genome sequence, we evaluated the runtime and the memory usage of the Big-LD algorithm applied to the entire 1000G dataset from chromosome 1 through 22 excluding only monomorphic SNPs. We averaged the runtime and the memory usage assessment over five repetitions of Big-LD executions for (1) Sub-task partitioning stage using SNPs with MAF ≥ 0.05, (2) Initial LD block partitioning stage using SNPs with MAF ≥ 0.05, (3) Entire LD block partitioning including the refinement steps for low frequency SNPs (Supplementary Fig. S4 and Table S6). The average runtime required to partition the entire dataset into sub-tasks of the limit size 1500 is 0.73 h and the maximum memory usage is 3584 Mb. The average runtime and the maximum memory usage until the initial LD block partitioning (including sub-task partitioning step) using only common SNPs (MAF ≥ 0.05; 5 429 840 SNPs) is 4.45 h and 4144MB. It takes 5.83 h to obtain LD blocks for the entire 1000G dataset of chromosome 1 through 22 excluding only monomorphic SNPs (13 288 240 SNPs) with maximum memory usage of 8828MB. We confirmed that runtime and memory usage increase linearly in most cases as the number of SNPs increases, which was expected, since Big-LD adopts a process of dividing the data into sub-tasks when massive SNP data are given (Supplementary Fig. S4).

We then compared the runtime and the memory usage of the Big-LD, S-MIG++ and MIG++ using the chromosome 22 data of 1000G dataset (Supplementary Table S7). For the experimental data, the average runtime of Big-LD is lowest (148.8 s for the data with MAF ≥ 0.05 and 204.6 s for the data with MAF > 0), which is a significant improvement over the existing methods MIG++ (5788.1 s for the data with MAF ≥ 0.05 and 57 743.2 s for the data with MAF > 0) and S-MIG++ (6074.3 s for the data with MAF ≥ 0.05 and 35 640.8 s for the data with MAF > 0). We did not include Haploview and MATILDE in this comparison since the runtime for these programs is much longer than that of the other methods. For example, to run MATILDE for a set of only 1000 SNPs, it takes about 4.33 h, and to run Haploview (CI, FGT, SS) for a set of 1500 SNPs, it takes about 0.17 h.

3.2 Comparisons of Big-LD block partition results with pre-existing methods

Using chromosome 22 data of the 1000G dataset and HapMap dataset, we compared LD block partition results of Big-LD with those produced by MATILDE (Pattaro *et al.*, 2008), Haploview (CI, FGT, SS) (Barrett *et al.*, 2005), MIG++ (Taliun *et al.*, 2014) and S-MIG++ (Taliun *et al.*, 2016). Due to runtime and memory limitations of Haploview program, we applied the Haploview computation to data divided into windows of 1500 SNPs shifted every 1000 SNPs and combined the results with decision rules for the overlapped regions that give priority to the larger block among two results. For MATILDE, we divided the data into windows of 1000 SNPs and obtained results for each window. We also ran the program for 500 SNPs located around each of the break points between windows and then combined results for the 1000 and 500 SNP windows similarly to the rules to combine Haploview results. For S-MIG++, we used the program for single CPU, and the sampling ratio was set as 0.01 and the probability of the correctly estimated upper limits for haplotype blocks boundaries was set as 0.99 (both are default values in the program).

In Supplementary Table S8, we summarize several characteristics of the LD block partition results obtained by the Big-LD and the other methods, including the total number of blocks produced, the average length of blocks meaning the difference in the base pair (bp) position between the first and the last SNPs in the block, the average number of SNPs per block, the average r^2 over all the pairs of SNPs in a block, and the average r^2 over all the SNP pairs, each of which belongs to two consecutive blocks. Overall, the Big-LD produces fewer blocks of larger size compared to the other methods. The size discrepancy of LD blocks between the results of Big-LD and the other methods is larger for 1000G data than for HapMap data. The mean of pairwise r^2 values within an LD block produced by Big-LD is slightly lower (0.472 for 1000G and 0.491 for HapMap) than the means for S-MIG++ (for 1000G) and Haploview-CI but higher than the means for MATILDE, MIG++, S-MIG++ (for HapMap),

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

Haploview-FGT and Haploview-SS. Especially, the mean of pairwise r^2 values within an LD block produced by Big-LD is much higher than that of MATILDE (0.491 vs 0.285) for HapMap. On the other hand, the average pairwise r^2 values across two consecutive LD blocks obtained by Big-LD is lowest (0.155 for 1000G and 0.141 for HapMap) compared to the other LD block construction methods except MATILDE. The distribution of pairwise r^2 values across two consecutive LD blocks obtained by Big-LD shows that more than 50% of the values are under 0.1 (Supplementary Fig. S5). The average pairwise D′ values within a LD block is lower than that of the other methods, moreover the average pairwise D′ values across consecutive blocks is much lower than that of the other methods (Supplementary Table S8). The distribution of pairwise D′ values across two consecutive blocks excluding singleton blocks shows that the percentage of the values between 0.85 and 0.95 obtained from the results of Big-LD is much less than the percentage obtained from the other methods (Supplementary Fig. S6).

In Figure 2, distributions of block lengths are plotted for all LD block partition methods. Supplementary Table S9 reports distributions of the number of SNPs per block for all LD block partition methods. The LD blocks obtained from the 1000G dataset by Haploview-CI, MIG++ and S-MIG++, which are based on the LD block definition of Gabriel *et al.* (2002), show similar block size distributions. Compared to the other methods, Haploview-FGT produces more blocks of length up to about 10k. Big-LD produces fewer blocks in size groups up until a certain size (20k for 1000G and 15k for HapMap) compared to the other LD block partition methods. MATILDE produces more LD blocks than Big-LD for 1000G, but produces fewer blocks than Big-LD for HapMap. Furthermore, especially for HapMap, Big-LD produces the most LD blocks of size at least 30 SNPs compared to the other methods except MATILDE.

In Figure 3, separate LD heatmaps for an example region on chromosome 22 show LD block boundaries obtained by all methods. This region exemplifies the case where a big LD block found by Big-LD is split into several small blocks by MATILDE, Haploview, MIG++ and S-MIG++. The big LD block between 23 250 737 bp and 23 644 677 bp position found by Big-LD is split into 6–22 blocks by the other methods for 1000G data and 2–7 blocks for HapMap data. The average r^2 values between the split consecutive blocks by the other methods is between 0.40 and 0.66 for 1000G data and between 0.54 and 0.65 for HapMap data (0.19 for MATILDE). In this example region, LD blocks found in 1000G data by Big-LD are also observed at similar locations in the HapMap data while there is greater discrepancy between 1000G and HapMap data results for the other LD block partition methods. The LD heatmaps of additional example regions including the top 30 big LD blocks obtained for 1000G and HapMap data are presented in Supplementary Figure S7, and we observed similarly that Big-LD produces large blocks which are usually split by the other methods. The LD blocks found by Big-LD in the two datasets also exhibit greater similarity compared to the other methods.

In Figure 4, we plot the distributions of LD block lengths obtained from 1000G and HapMap data together for each LD block partition method. In Big-LD, the number of small blocks of up to 2 kb is usually larger in 1000G data compared to HapMap data while the numbers of blocks greater than 2 kb are usually similar in both datasets. The other LD block partition methods produce more blocks of up to 20 kb from 1000G data than from HapMap data, suggesting the differences in block partition results for the datasets with different density is greater in the other methods than in Big-LD. We also examined actual overlap proportions of LD blocks

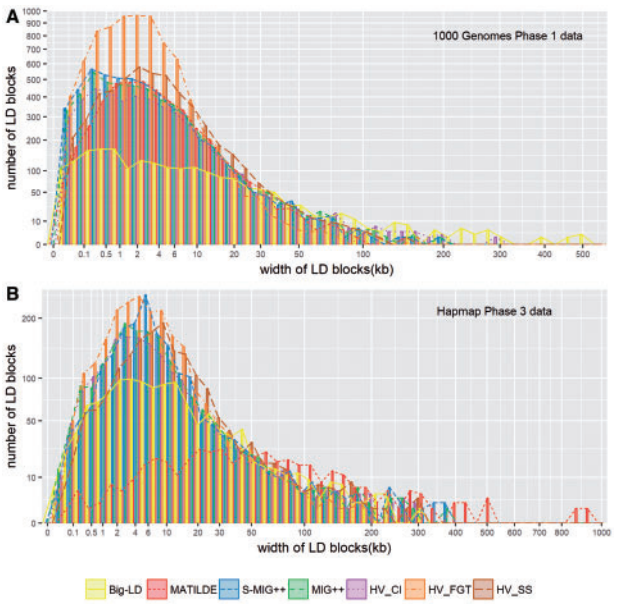


Fig. 2. Distributions of the length of LD block obtained using chromosome 22 region of (**A**) 1000G dataset and (**B**) HapMap dataset. The length of the LD block is obtained as the difference of the bp position of starting and ending SNPs of a LD block

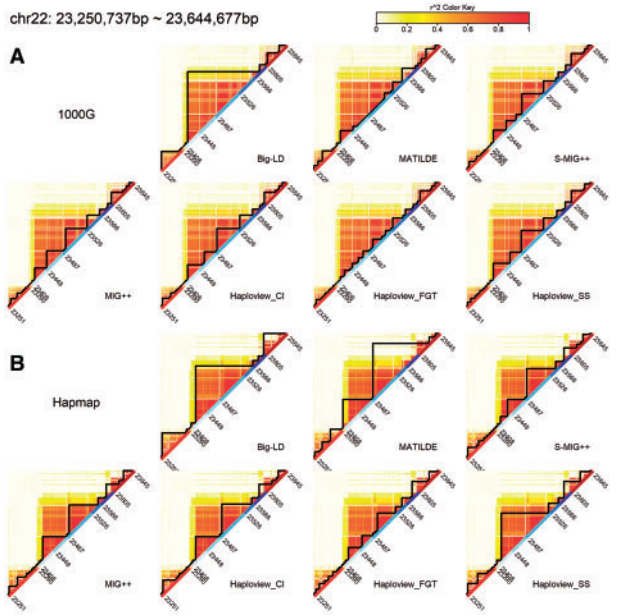


Fig. 3. An example of LD heatmaps of the region chr22: 23 251–23 645 kb with LD block partition results obtained from (**A**) 1000G dataset and (**B**) HapMap dataset. A solid triangle represents each LD block. Selected bp position in kb are shown

obtained from both datasets for each method (Supplementary Table S10). The proportion of overlapping LD blocks in each dataset over the number of all LD blocks produced in each dataset is larger for Big-LD compared to the other methods. This tendency is even stronger when comparing only the larger blocks. For example, when we use the overlap declaration criterion of 80% and limit the comparison to the blocks greater than 10 kb, the overlap proportions of LD blocks produced by Big-LD are 30% of the number of such blocks

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

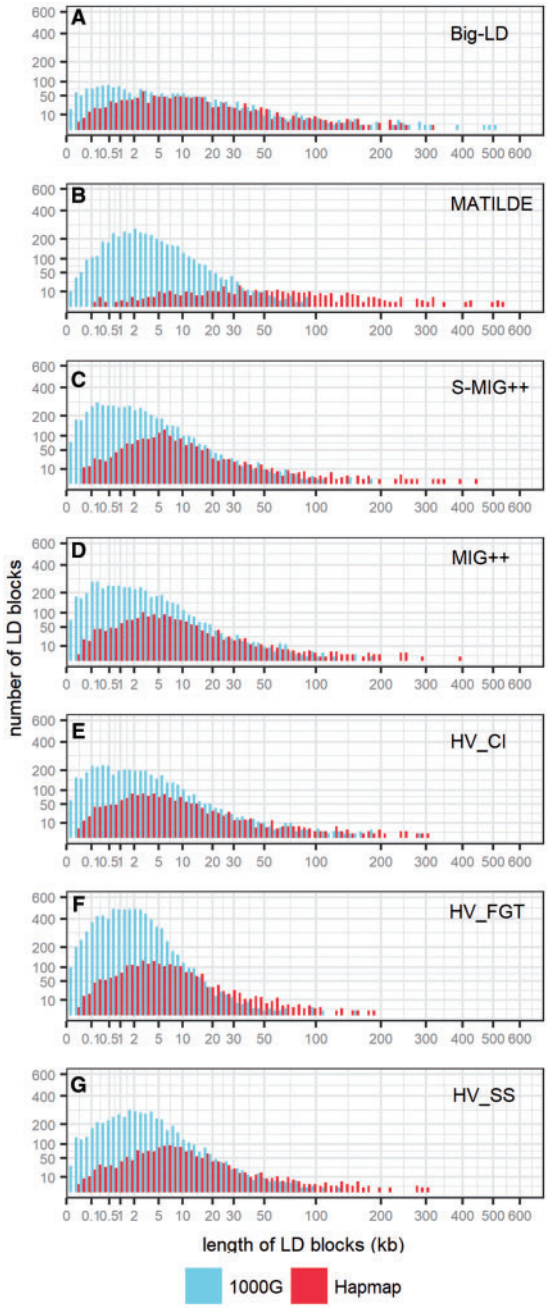
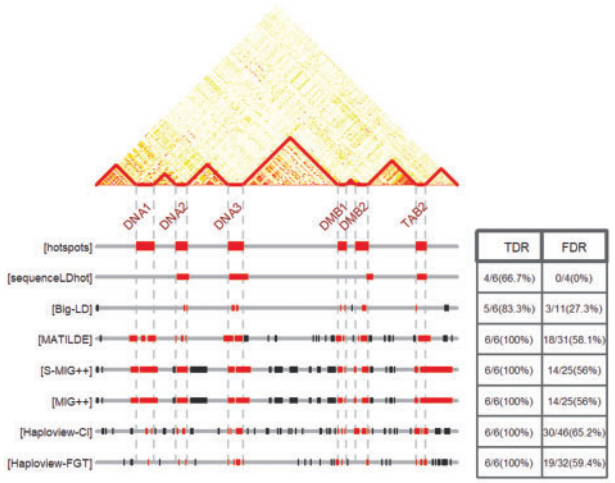


Fig. 4. The distributions of the length of LD blocks obtained from 1000G and HapMap chromosome 22 datasets for each method of (**A**) Big-LD, (**B**) MATILDE, (**C**) S-MIG++, (**D**) MIG++, (**E**) Haloview (CI), (**F**) Haploview (FGT) and (**G**) Haploview (SS). The length of the LD block is obtained as the difference of the bp position of starting and ending SNPs of a LD block

in 1000G data and 35% of the number of blocks in HapMap data while these overlap proportions are 1–29% for the other methods.

We also evaluated haplotype diversity measured by the haplotype diversity index for a block (Patil *et al.* 2001; Zhang *et al.* 2002). The *haplotype diversity index* is defined as the ratio of the number of common haplotypes in the block over the total number of haplotypes within the block. The mean haplotype diversity indices of the blocks obtained by Big-LD for 1000G and HapMap are 0.911 and 0.985, respectively, which are lower compared to those obtained by the other methods except MATILDE (0.87 for HapMap) (Supplementary Table S11). The distribution of haplotype diversity indices shows that the haplotype diversity of the blocks obtained by Big-LD decreases as the LD block size increases (Supplementary Fig. S8).



TDR	FDR
4/6(66.7%)	0/4(0%)
5/6(83.3%)	3/11(27.3%)
6/6(100%)	18/31(58.1%)
6/6(100%)	14/25(56%)
6/6(100%)	14/25(56%)
6/6(100%)	30/46(65.2%)
6/6(100%)	19/32(59.4%)

Fig. 5. The LD heatmap of the MHC dataset with true recombination hotspot positions found by sperm-typing experiments in Jeffreys *et al.* (2001), LD block boundary locations found by Big-LD, MATILDE, Haploview (CI, FGT, SS), MIG++ and S-MIG++ and hotspot locations found by sequenceLDhot. The red bars show LD block boundaries overlapping with true recombination hotspots and the black bars show LD block boundaries not overlapping with them. TDR is the number of the true recombination hotspots overlapping with the block boundaries found by each method over the number of true recombination hotspots which is 6, and FDR is the number of non-overlapping block boundaries with the true recombination hotspots among the ones found by each method over the all block boundaries found by each method

3.3 LD block and recombination hotspots

For the MHC dataset of 50 north-European British semen donors, we obtained the LD block partition results using Big-LD, MATILDE, three Haploview methods (CI, FGT, SS), MIG++ and S-MIG++ to compare with the true recombination hotspot locations obtained by sperm-typing experiment on the same sample of individuals in the MHC dataset reported in Jeffreys *et al.* (2001). For this comparison, we regarded the regions between non-singleton LD blocks as the block boundary regions. Also, we obtained estimated hotspots from MHC data using sequenceLDhot (Fearnhead, 2006) to compare with the true recombination hotspot regions. For sequenceLDhot, we set the criteria to declare hotspots when the log-likelihood ratio over the background recombination rate is greater than 12. Also, we set the sliding window option for sequenceLDhot as 2 kb windows moving about every 1 kb point. In Figure 5, the LD heatmap of this region and the block boundary locations or hotspot locations found by all methods are shown along with the true discovery rate (TDR) defined as the ratio of the number of block boundaries or hotspots coinciding with the true hotspots over the number of all true hotspots found by the sperm-typing experiment (total of 6 hotspots). The false discovery rate (FDR) is defined as the ratio of the number of block boundaries or hotspots not in the true hotspot regions over the number of all block boundaries or hotspots found by the given method. Big-LD finds five out of six true hotspots (TDR of 83.3%) and shows low FDR (27.3%). The other LD block partition methods including the MATILDE, three Haploview methods and MIG++ yield 100% TDR but also show high FDRs greater than 50%. The sequenceLDhot yields lower TDR (66.7%) than the Big-LD, but shows zero FDR.

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

We also compared LD block partition results of Big-LD and the other methods applied to 1000G and HapMap data with the recombination hotspot locations obtained for these datasets by sequenceLDhot to see if the LD block boundaries and estimated recombination hotspot regions coincide with each other. Considering the recombination hotspot regions estimated by the sequenceLDhot as the true hotspot regions, we obtained TDR and FDR similarly defined for the MHC dataset analysis (Supplementary Table S12). The TDR for Big-LD results (65.5% in 1000G and 85.8% in HapMap) is lowest compared to the TDR for the other methods (67–99%) except MATILDE (40% in HapMap) since the other methods produced smaller blocks with more chance to coincide with the estimated recombination hotspot regions. However, the FDR for Big-LD (65.2% for 1000G and 73.4% for HapMap) is lowest among all LD block partition methods except MATILDE (66.5% in HapMap). On average, the distance between consecutive estimated recombination hotspots is 46 699 kb (99.5 SNPs) for 1000G data and 94 802 kb (38.5 SNPs) for HapMap data, which are twice and four times the distance between consecutive block boundary positions obtained by Big-LD for 1000G and HapMap data respectively. LD heatmaps of some example regions compare the block boundaries of all partition methods and estimated recombination hotspot positions obtained by the sequenceLDhot (Supplementary Fig. S9).

4 Discussion

Big-LD occupies much more memory (at most 3042 Mb for 1000G chromosome 22 data) compared to S-MIG++ (less than 25 Mb for for 1000G chromosome 22 data) mostly due to the clique finding algorithms in R *igraph* package. However, Big-LD can run without problems on high performance computers with more than 4GB memory.

The LD block partition methods adopting a narrow sense definition of LD blocks tend to separate SNPs that are in strong LD if they have some SNPs in between that are in low LD with those 'strong LD' SNPs. This case is observed more often when more SNPs are genotyped in the same region such as with sequencing data. As a result, the existing LD block construction algorithms usually split large LD block into small blocks as evident in the application to the 1000G dataset. For Big-LD, the problem of 'interruption by low LD SNPs' is less likely to affect the LD block construction results because Big-LD uses an agglomerative approach that starts by identifying small communities of SNPs, i.e. the SNPs in each LD bin region (from starting SNP to ending SNP), and proceeds by merging these communities. By including all SNPs that are located between starting and ending SNPs of each LD bin in the initial communities of SNPs, low LD SNPs in the middle of a high LD region do not severely affect the block partition results. Allowing this region of low LD between strongly associated SNP pairs goes against the criteria for evaluation of haplotype block construction results suggested by Wall and Pritchard (2003): 'absence of holes' (If a pair of SNPs are in strong LD, then both SNPs should be in strong LD with SNPs that lie in between). When the purpose of haplotype block construction is to find tag SNPs, the presence of holes in the block region can yield cases where tag SNPs are not capturing the effects of some SNPs in the block. When, however, the purpose of haplotype block construction is to find blocks with low between-block correlation or blocks revealing population genetic structure such as recombination hotspots, especially with high density sequencing data, the 'absence of holes' principle may not serve the purpose.

We found that SNPs in the LD blocks produced by Big-LD show higher r^2 and D′ values when they are in the same block and lower r^2 and D′ values when they are in different blocks. In the three Haploview methods, as well as in MIG++, S-MIG++ and MATILDE, SNPs in the same block have similar, slightly higher or lower r^2 and D′ values compared to Big-LD, but SNPs in consecutive blocks also have higher r^2 and D′ values. This result shows that Big-LD produces more optimized blocks compared to the other methods in the sense that r^2 values are high within the same block and low between different blocks. In particular, if we want to use LD block construction results to choose SNP sets for genetic association analysis using multi-SNP methods where the effects of multiple SNPs are combined, low r^2 values between different SNP sets would result in less correlated null statistics corresponding to multiple hypotheses. To adjust p-values of the association analysis results by the genomic significance level using Bonferroni method, the independence of null hypotheses representing many multi-SNP statistics is needed (McDonald, 2009). In our evaluation of the power of various multi-SNP association tests based on various LD block construction results, linear combination type or marginal SNP score based tests usually show better power for big LD block region produced by Big-LD under multiple causal SNP models, whereas the power of quadratic sum type tests was lower when the effects of multiple SNPs are quadratically combined over a large block (Supplementary Results; Supplementary Tables S13 and S14).

When we compared the LD block partition results obtained using 1000G and HapMap datasets, we observed that the LD blocks found by Big-LD from two datasets overlap more than the other methods, especially for large size blocks. This result suggests that Big-LD finds more invariant, less data-dependent big LD blocks regardless of the marker-density of the data. Also, the difference between the LD block construction results of Big-LD and the other methods was even greater in 1000G data than in HapMap data. With the dense SNPs in the 1000G data, CLQ-D usually produces a block LD structure with more LD bins that consist of non-consecutive SNPs compared to HapMap data (Fig. 3). Since the ranges of LD bins are more invariant to the marker-density of genotyped SNPs, Big-LD produces more invariant big LD blocks by the 'bottom-up' type clustering of LD bins. For the other methods such as MATILDE, Haploview, S-MIG++ or MIG++, the block partition results are very sensitive to the SNP marker-density and/or the selection of SNPs.

How the locations of recombination hotspots are related to LD blocks is of major interest since a recombination hotspot is the one of the main reasons for LD block formation. In early days, without clear operational definitions for LD block, researchers found by observing some regions that true recombination hotspot locations confirmed by sperm-typing experiments coincide with LD block boundaries found as LD break points (Jeffreys *et al.*, 2001). After several LD block definitions were introduced, other researchers found that LD blocks defined in a narrow sense can be formed without recombination hotspots (Wang *et al.*, 2002). In our comparisons of LD block partition results obtained by Big-LD algorithm and the other methods with the experimentally determined recombination hotspots for the same MHC dataset used in Jeffreys *et al.* (2001), we found that LD blocks found by Big-LD agree better with the true recombination hotspots than the existing methods. We also saw better agreement between the LD block locations found by Big-LD and the estimated hotspot regions obtained by the sequenceLDhot compared to the other LD block partition methods. However, the agreement proportions between the results of Big-LD and sequenceLDhot do not exceed 50% even with relaxed criterion such as 60% overlap to

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

be declared as a shared LD block. Since the estimated recombination hotspots by sequenceLDhot for MHC data also did not coincide with the real hotspots perfectly (lower TDR than Big-LD), more comprehensive data with accurate information on hotspots for large regions would be helpful to further determine the relationship between the LD blocks found by Big-LD and recombination hotspots.

Most graph clustering algorithms require presetting the number of clusters before applying clustering steps (de Amorim and Hennig, 2015). Big-LD does not require prior specification of the number of clusters. Instead, the clusters are chosen at each step of the greedy process which picks out a set of independent cliques of the interval graph that do not overlap each other. It uses an algorithm to choose the maximum weight independent set of a graph where the vertices are all cliques in the interval graph and the edges represent the overlap between the intervals covered by each clique. By applying this process until the remaining genomic regions are all covered by non-overlapping intervals, i.e. the LD blocks, Big-LD can automatically determine how many clusters should be present in the final partitioning results.

In summary, the new LD block construction method Big-LD produces larger LD blocks than existing methods since it adopts a wide sense definition for LD block, which allows a 'hole' of low LD SNPs to be present between strongly associated SNP pairs. Big-LD determines the number of blocks using a maximum weight independent set selection algorithm. The LD blocks produced by Big-LD agree better than existing methods with the recombination hotspot locations determined by sperm-typing experiments. The observed average runtime of Big-LD for 13 288 240 non-monomorphic SNPs from 1000 Genomes Project autosome data (286 East Asians) is about 5.83 h, which is a significant improvement over the existing methods.

Funding

This work was supported by the National Research Foundation of Korea (NRF) [grant numbers NRF-2012R1A1A3012428, NRF-2015R1A1A3A04001269, NRF-2015R1A2A2A01006885]; and the Canadian Institutes of Health Research operating [grant number MOP-84287].

Conflict of Interest: none declared.

References

Barrett,J.C. *et al.* (2005) Haploview: analysis and visualization of LD and haplotype maps. *Bioinformatics*, **21**, 263–265.

Bron,C. and Kerbosch,J. (1973) Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM*, **16**, 575–577.

Collins,A. *et al.* (1999) Genetic epidemiology of single-nucleotide polymorphisms. *Proc. Natl. Acad. Sci. USA*, **96**, 15173–15177.

Csardi,G. and Nepusz,T. (2006) The igraph software package for complex network research. *InterJournal*, Complex Systems, 1695.

Daly,M.J. *et al.* (2001) High-resolution haplotype structure in the human genome. *Nat. Genet.*, **29**, 229–232.

de Amorim,R.C. and Hennig,C. (2015) Recovering the number of clusters in data sets with noise features using feature rescaling factors. *Inf. Sci.*, **324**, 126–145.

Derkach,A. *et al.* (2014) Pooled association tests for rare genetic variants: a review and some new results. *Stat. Sci.*, **29**, 302–321.

Eppstein,D. *et al.* (2010) Listing all maximal cliques in sparse graphs in near-optimal time. In: *International Symposium on Algorithms and Computation*. Springer, Berlin, Heidelberg, pp. 403–414.

Fearnhead,P. (2006) SequenceLDhot: detecting recombination hotspots. *Bioinformatics*, **22**, 3061–3066.

Gabriel,S.B. *et al.* (2002) The structure of haplotype blocks in the human genome. *Science*, **296**, 2225–2229.

Gibbs,R.A. *et al.* (2003) The International HapMap project. *Nature*, **4**, 789–796.

Goddard,K.A.B. *et al.* (2000) Linkage disequilibrium and allele-frequency distributions for 114 single-nucleotide polymorphisms in five populations. *Am. J. Hum. Genet.*, **66**, 216–234.

Jeffreys,A.J. *et al.* (2000) High resolution analysis of haplotype diversity and meiotic crossover in the human TAP2 recombination hotspot. *Hum. Mol. Genet.*, **9**, 725–733.

Jeffreys,A.J. *et al.* (2001) Intensely punctate meiotic recombination in the class II region of the major histocompatibility complex. *Nat. Genet.*, **29**, 217–222.

Lewontin,R.C. (1964) The interaction of selection and linkage. I. General considerations; heterotic models. *Genetics*, **49**, 49–67.

Lewontin,R.C. (1995) The detection of linkage disequilibrium in molecular sequence data. *Genetics*, **140**, 377–388.

Manolio,T.A. (2010) Genomewide association studies and assessment of the risk of disease. *N. Engl. J. Med.*, **363**, 166–176.

McDonald,J.H. (2009) *Handbook of Biological Statistics. Vol. 2*. Sparky House Publishing, Baltimore, MD.

McVean,G.A.T. *et al.* (2004) The fine-scale structure of recombination rate variation in the human genome. *Science*, **304**, 581–584.

Neale,B.M. *et al.* (2011) Testing for an unusual distribution of rare variants. *PLoS Genet.*, **7**, e1001322.

Neale,B.M. and Sham,P.C. (2004) The future of association studies: gene-based analysis and replication. *Am. J. Hum. Genet.*, **75**, 353–362.

Ott,J. (2000) Predicting the range of linkage disequilibrium. *Proc. Natl. Acad. Sci. USA*, **97**, 2–3.

Pan,W. (2009) Asymptotic tests of association with multiple SNPs in linkage disequilibrium. *Genet. Epidemiol.*, **33**, 497–507.

Patil,N. (2001) Blocks of limited haplotype diversity revealed by high-resolution scanning of human chromosome 21. *Science*, **294**, 1719–1723.

Pattaro,C. *et al.* (2008) Haplotype block partitioning as a tool for dimensionality reduction in SNP association studies. *BMC Genomics*, **9**, 405.

Rana,N.A. *et al.* (2004) Recombination hotspots and block structure of linkage disequilibrium in the human genome exemplified by detailed analysis of PGM1 on 1p31. *Hum. Mol. Genet.*, **13**, 3089–3102.

Reich,D.E. *et al.* (2001) Linkage disequilibrium in the human genome. *Nature*, **411**, 199–204.

Sabeti,P.C. *et al.* (2007) Genome-wide detection and characterization of positive selection in human populations. *Nature*, **449**, 913–918.

Slatkin,M. (2008) Linkage disequilibrium—understanding the evolutionary past and mapping the medical future. *Nat. Rev. Genet.*, **9**, 477–485.

Stephens,M. and Scheet,P. (2005) Accounting for decay of linkage disequilibrium in haplotype inference and missing-data imputation. *Am. J. Hum. Genet.*, **76**, 449–462.

Stephens,M. *et al.* (2001) A new statistical method for haplotype reconstruction from population data. *Am. J. Hum. Genet.*, **68**, 978–989.

Takahata,N. (1982) Linkage disequilibrium, genetic distance and evolutionary distance under a general model of linked genes or a part of the genome. *Genet. Res.*, **39**, 63–77.

Taliun,D. *et al.* (2014) Efficient haplotype block recognition of very long and dense genetic sequences. *BMC Bioinf.*, **15**, 10.

Taliun,D. *et al.* (2016) Fast sampling-based whole-genome haplotype block recognition. *IEEE/ACM Trans. Comput. Biol. Bioinf.*, **13**, 315–325.

The International HapMap 3 Consortium (2010) Integrating common and rare genetic variation in diverse human populations. *Nature*, **467**, 52–58.

The 1000 Genomes Project Consortium (2012) An integrated map of genetic variation from 1, 092 human genomes. *Nature*, **491**, 56–65.

Tsukiyama,S. *et al.* (1977) A new algorithm for generating all the maximal independent sets. *SIAM J. Comput.*, **66**, 505–517.

Twells,R.C. *et al.* (2003) Haplotype structure, LD blocks, and uneven recombination within the LRP5 gene. *Genome Res.*, **13**, 845–855.

Wall,J.D. and Pritchard,J.K. (2003) Assessing the performance of the haplotype block model of linkage disequilibrium. *Am. J. Hum. Genet.*, **73**, 502–515.

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023

Wang,N. *et al*. (2002) Distribution of recombination crossovers and the origin of haplotype blocks: the interplay of population history, recombination, and mutation. *Am. J. Hum. Genet*., **71**, 1227–1234.

Wang,Q. *et al*. (2015) A review of study designs and statistical methods for genomic epidemiology studies using next generation sequencing. *Front. Genet*., **6**, 149.

Wu,M.C. *et al*. (2010) Powerful SNP-set analysis for case-control genome-wide association studies. *Am. J. Hum. Genet*., **86**, 929–942.

Yoo,Y.J. *et al*. (2015) Clique-based clustering of correlated SNPs in a gene can improve performance of gene-based multi-bin linear combination test. *BioMed Res. Int*., **2015**, 852341.

Yoo,Y.J. *et al*. (2017) Multiple-linear-combination (MLC) regression tests for common variants adapted to linkage disequilibrium structure. *Genet. Epidemiol*., **41**, 108–121.

Zapata,C. (2011) On the uses and applications of the most commonly used measures of linkage disequilibrium from the comparative analysis of their statistical properties. *Hum. Hered*., **71**, 186–195.

Zhang,K. *et al*. (2002) A dynamic programming algorithm for haplotype block partitioning. *Proc. Natl. Acad. Sci. USA*, **99**, 7335–7339.

Zhao,J. *et al*. (2011) Pathway-based analysis using reduced gene subsets in genome-wide association studies. *BMC Bioinf*., **12**, 1–14.

Downloaded from https://academic.oup.com/bioinformatics/article/34/3/388/4282661 by INRA Avignon user on 17 August 2023